Kubernetes

Fundamentals Guide

From architecture to autoscaling, explained with simple examples for beginners and aspiring DevOps engineers

WHAT YOU WILL LEARN

01 Architecture: control plane and worker nodes
02 Pods, containers, lifecycle and networking
03 Namespaces, labels and selectors
04 Services: all four types
05 ReplicationController, ReplicaSet, Deployment
06 Deployment strategies
07 StatefulSet and DaemonSet
08 Node affinity, taints and tolerations
09 ConfigMap and Secret
10 PV, PVC and StorageClass
11 Readiness, Liveness and Startup probes
12 Ingress and Ingress Controller
13 HPA, VPA and Cluster Autoscaler

Contents

1. Kubernetes Architecture . . . 3

2. Pods, Containers, Lifecycle and Networking . . . 6

3. Namespaces, Labels and Selectors . . . 10

4. Services . . . 13

5. ReplicationController, ReplicaSet and Deployment . . . 16

6. Deployment Strategies . . . 19

7. StatefulSet and DaemonSet . . . 22

8. Node Affinity, Taints and Tolerations . . . 25

9. ConfigMap and Secret . . . 28

10. PV, PVC, StorageClass and Provisioning . . . 31

11. Readiness, Liveness and Startup Probes . . . 34

12. Ingress, Ingress Controller and Ingress Rules . . . 37

13. HPA, VPA and Cluster Autoscaler . . . 40

14. Putting It All Together . . . 43

How to use this guide

Each chapter starts with a plain-English explanation and a real-life **analogy**, then shows a small YAML example and the `kubectl` commands to try it. Practise every example on a local cluster (minikube, kind or Docker Desktop) - typing the commands yourself is the fastest way to learn.

1 Kubernetes Architecture

Control plane, worker nodes and how they cooperate

Kubernetes (often written **K8s**) is an open-source system that runs and manages containers across many machines. You tell it *what you want* (for example, "run 3 copies of my web app") and it continuously works to make that true: starting containers, replacing crashed ones, spreading load and rolling out updates.

> **Real-life analogy**
> Think of a large **restaurant**. The **control plane** is the management office: it takes orders, decides which kitchen station cooks what and keeps the master list of everything happening. The **worker nodes** are the kitchen stations where the cooking (running your containers) actually happens. Kubernetes is the manager who makes sure every order is cooked, even when a cook calls in sick.

1.1 The big picture: a cluster

A **cluster** is a set of machines (physical or virtual) split into two roles:

- **Control plane** - the brain. It makes decisions (scheduling, scaling, reacting to failures) and stores the cluster's state.
- **Worker nodes** - the muscle. They run your application containers inside Pods.

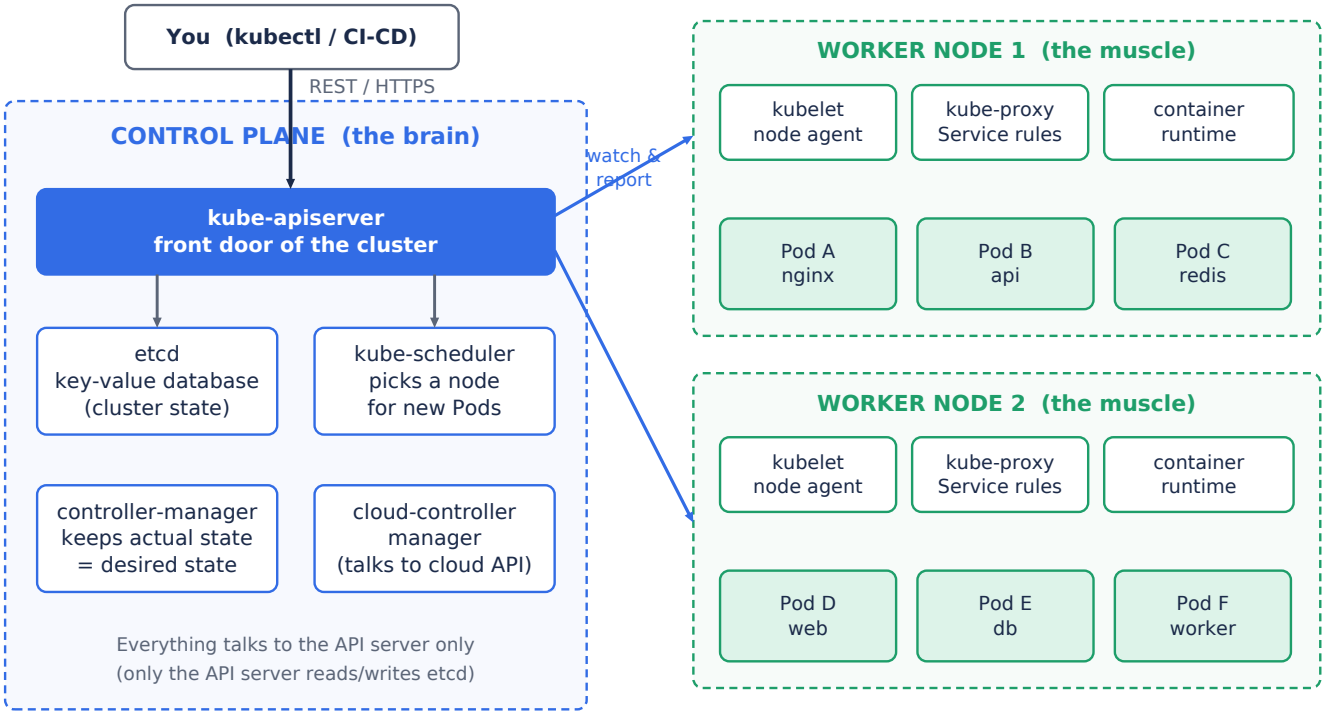


Figure 1.1 - A Kubernetes cluster: one control plane managing several worker nodes

1.2 Control plane components

Component	What it does	Easy way to remember
`kube-apiserver`	The single entry point. Every command (`kubectl`, dashboards, other components) goes through its REST API. It validates requests and is the only component that reads/writes etcd.	The reception desk
`etcd`	A highly available key-value database holding the entire cluster state: what Pods exist, their configs, Secrets, etc. If etcd is lost, the cluster's memory is lost - back it up.	The filing cabinet
`kube-scheduler`	Watches for new Pods with no node assigned and picks the best node, based on free CPU/memory, affinity rules, taints and more.	The seating host
`kube-controller-manager`	Runs many small control loops (controllers): Node controller, ReplicaSet controller, Deployment controller, etc. Each compares *desired* vs *actual* state and fixes differences.	The supervisors
`cloud-controller-manager`	Connects Kubernetes to your cloud provider (creates load balancers, attaches disks, detects deleted VMs). Absent on bare-metal clusters.	The cloud liaison

1.3 Worker node components

Component	What it does
`kubelet`	The agent on every node. It receives Pod specs from the API server, tells the container runtime to start containers, runs health probes and reports status back.
`kube-proxy`	Maintains network rules (iptables / IPVS) on the node so that traffic sent to a **Service** reaches the right Pods.
Container runtime	The software that actually runs containers, for example **containerd** or **CRI-O**. Kubernetes talks to it through the Container Runtime Interface (CRI).

1.4 What happens when you create a Deployment?

Let's trace `kubectl create deployment web --image=nginx --replicas=3` step by step:

1. `kubectl` sends the request to the **API server**, which authenticates it and saves the Deployment object in **etcd**.
2. The **Deployment controller** notices the new Deployment and creates a **ReplicaSet**.
3. The **ReplicaSet controller** notices it has 0 of 3 Pods and creates 3 Pod objects (still not assigned to any node).
4. The **scheduler** sees 3 unassigned Pods and assigns each to a suitable node.
5. The **kubelet** on each chosen node sees a Pod assigned to it, asks the runtime to pull the image and start the container.
6. The kubelet keeps reporting status; the controllers keep watching. If a Pod dies, the loop repeats until 3 are running again.

> **Desired state vs actual state**
> Kubernetes is **declarative**. You describe the *desired state* in YAML and Kubernetes keeps working (the **reconciliation loop**) until the *actual state* matches. You rarely say "start this container"; you say "there should be 3 of these".

1.5 Try it yourself

Terminal

```
kubectl cluster-info                # address of the API server and CoreDNS
kubectl get nodes -o wide           # list nodes, their roles, IPs and versions
kubectl get pods -n kube-system     # control plane + system Pods (coredns, kube-proxy ...)
kubectl describe node <node-name>   # capacity, conditions, running Pods on that node
kubectl api-resources               # every object type this cluster understands
```

Tip

Local practice: install **minikube**, **kind** or **Docker Desktop** Kubernetes to get a free single-machine cluster. In the cloud (EKS, AKS, GKE) the provider runs and upgrades the control plane for you; in production self-managed clusters run **3 control plane nodes** for high availability.

2 Pods, Containers, Lifecycle and Networking

The smallest deployable unit in Kubernetes

A **container** packages an application with everything it needs. Kubernetes does not run bare containers; it wraps one or more tightly-coupled containers in a **Pod**. The Pod is the smallest unit you can create, schedule and scale.

Real-life analogy

A Pod is like a **small apartment** and containers are the **roommates**. They share one street address (IP address), can talk to each other by shouting across the room (`localhost`) and can share a cupboard (volumes). If the apartment is demolished, everyone inside goes with it.

2.1 Your first Pod

hello-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  labels:
    app: hello
spec:
  containers:
    - name: web
      image: nginx:1.27
      ports:
        - containerPort: 80
      resources:
        requests: { cpu: "100m", memory: "64Mi" }
        limits:   { cpu: "250m", memory: "128Mi" }
```

Every Kubernetes YAML has four top-level parts: **apiVersion** (which API group), **kind** (type of object), **metadata** (name, labels) and **spec** (what you want).

Terminal

```
kubectl apply -f hello-pod.yaml          # create it
kubectl get pods -o wide                 # status, Pod IP and node
kubectl describe pod hello-pod           # events: scheduled, pulled, started ...
kubectl logs hello-pod                   # container logs (-f to follow, -c <name> if many containers)
kubectl exec -it hello-pod -- sh         # open a shell inside the container
kubectl port-forward pod/hello-pod 8080:80   # reach it at http://localhost:8080
kubectl delete pod hello-pod
```

Requests vs limits

requests = what the scheduler reserves for the container (used to choose a node). **limits** = the maximum it may use. Exceed the memory limit and the container is killed (**OOMKilled**); exceed the CPU limit and it is only slowed down (throttled). `100m` means 100 millicores = 0.1 CPU.

2.2 Multi-container Pods

Usually one Pod = one main container. Add more containers only when they must live and die together and share resources. Common patterns:

Pattern	Purpose	Example
Sidecar	Helper that runs alongside the main app	Log shipper, service-mesh proxy
Init container	Runs to completion **before** the app starts	Wait for database, download config
Ambassador / adapter	Proxy or reformat traffic or metrics	Convert app metrics to Prometheus format

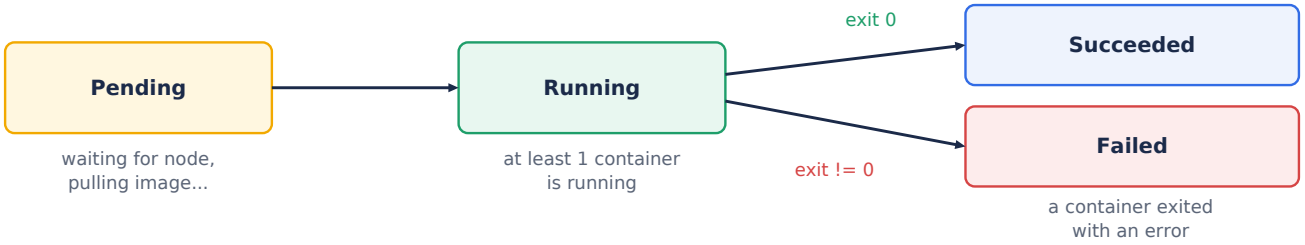
sidecar.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-sidecar
spec:
  volumes:
    - name: logs
      emptyDir: {}                  # scratch disk shared by both containers
  initContainers:
    - name: wait-for-db
      image: busybox:1.36
      command: ["sh", "-c", "until nslookup db; do echo waiting; sleep 2; done"]
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date >> /var/log/app.log; sleep 5; done"]
      volumeMounts: [{ name: logs, mountPath: /var/log }]
    - name: log-reader                 # the sidecar
      image: busybox:1.36
      command: ["sh", "-c", "tail -F /var/log/app.log"]
      volumeMounts: [{ name: logs, mountPath: /var/log }]
```

2.3 Pod lifecycle

A Pod moves through **phases**. Inside it, each container has a **state** (Waiting, Running or Terminated).

Pod phases



Deleting a Pod (graceful shutdown)



Figure 2.1 - Pod phases (top) and the graceful shutdown sequence (bottom)

Phase	Meaning
Pending	Pod accepted but not running yet: waiting for the scheduler, for volumes, or pulling the image.
Running	Pod is bound to a node and at least one container is running (or restarting).
Succeeded	All containers finished successfully (exit code 0) and will not restart. Typical for Jobs.
Failed	All containers have stopped and at least one ended in failure.

Phase	Meaning
Unknown	Pod state cannot be determined, usually because the node cannot be reached.

Restart policy

`spec.restartPolicy` decides what the kubelet does when a container exits: **Always** (default - restart no matter what, ideal for web servers), **OnFailure** (restart only on non-zero exit) or **Never** (used for one-off tasks).

Statuses you will see in kubectl get pods

Status	What it usually means	First thing to check
ContainerCreating	Image being pulled or volume being mounted	`kubectl describe pod`
ImagePullBackOff / ErrImagePull	Wrong image name/tag or no registry credentials	Image name, `imagePullSecrets`
CrashLoopBackOff	Container starts, crashes, restarts again with growing delay	`kubectl logs --previous`
OOMKilled	Container exceeded its memory limit	Raise memory limit or fix leak
Pending (long)	No node has enough resources, or taints/affinity block it	Events in `describe`
Completed	Container finished successfully (Job-style Pod)	Nothing - it is fine

2.4 Pod networking basics

Kubernetes networking follows a few simple rules:

- Every Pod gets its **own IP address**.
- Containers **inside one Pod** share that IP and talk via `localhost` (so two containers in a Pod cannot use the same port).
- Any Pod can reach any other Pod **directly by IP**, across nodes, **without NAT**. A **CNI plugin** (Calico, Cilium, Flannel, AWS VPC CNI...) makes this work.
- Pod IPs are **temporary**. When a Pod is replaced it gets a new IP - which is why we use **Services** (Chapter 4).
- **CoreDNS** gives names to Services, e.g. `my-service.my-namespace.svc.cluster.local`.

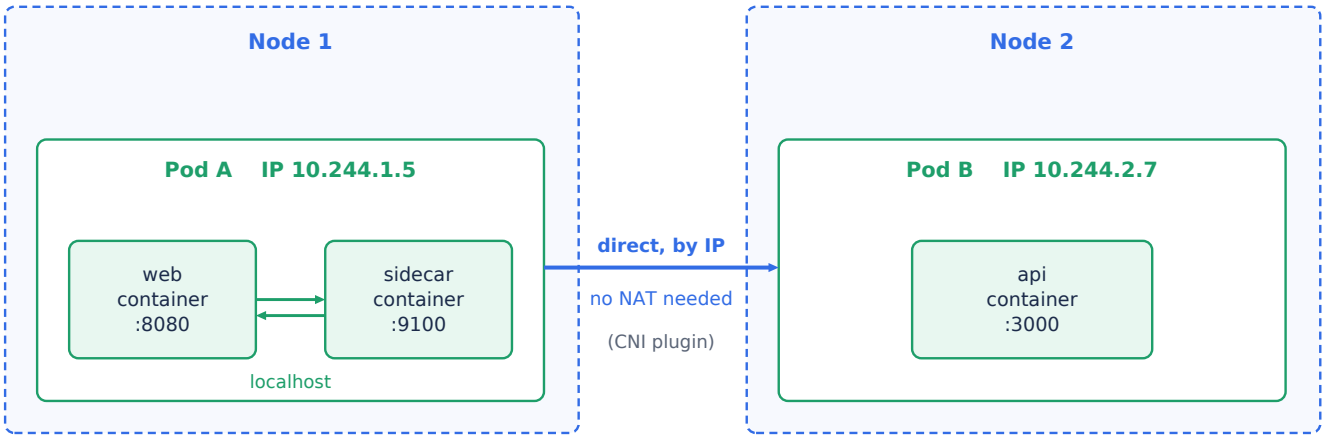


Figure 2.2 - Containers share their Pod's IP; Pods talk to each other directly

Who talks to whom	How
Container to container (same Pod)	`localhost:<port>`
Pod to Pod	Pod IP directly (but it changes - avoid hard-coding)
Pod to a group of Pods	Through a **Service** (stable IP + DNS name)
Outside world to Pods	**NodePort** / **LoadBalancer** / **Ingress** (Chapters 4 and 12)

Tip

Experiment: run `kubectl run client --image=busybox:1.36 -it --rm -- sh`, then `wget -qO- <pod-ip>` against another Pod's IP (see it with `kubectl get pods -o wide`).

3 Namespaces, Labels and Selectors

Organising, grouping and finding objects

3.1 Namespaces

A **namespace** is a virtual cluster inside your real cluster. It lets teams or environments (dev, test, prod) share one cluster without stepping on each other: names only need to be unique *within* a namespace, and you can attach quotas and access rules per namespace.

> **Real-life analogy**
> A namespace is like a **folder** on your computer or a **floor** in an office building. Two floors can each have a "Room 1" without confusion.

Built-in namespace	Purpose
`default`	Where objects go if you do not specify a namespace
`kube-system`	Kubernetes system components (CoreDNS, kube-proxy, ...)
`kube-public`	Readable by everyone; rarely used
`kube-node-lease`	Node heartbeat objects (health checks of nodes)

Terminal

```
kubectl create namespace dev                    # imperative way
kubectl get namespaces
kubectl apply -f app.yaml -n dev                # deploy into a namespace
kubectl get pods -n dev                         # list Pods in dev
kubectl get pods --all-namespaces               # (or -A) everything everywhere
kubectl config set-context --current --namespace=dev    # make dev your default
```

namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
apiVersion: v1
kind: ResourceQuota              # cap what the whole namespace may use
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    pods: "20"
    requests.cpu: "4"
    requests.memory: 8Gi
```

- A Service in the same namespace is reached by its short name: `http://backend`.
- From another namespace use the longer name: `http://backend.dev.svc.cluster.local`.
- Not everything is namespaced. **Nodes, PersistentVolumes, StorageClasses and Namespaces themselves** are cluster-wide.

> **Watch out**
> Namespaces are **not a security boundary by themselves**. Pods in different namespaces can still talk to each other unless you add **NetworkPolicies** and RBAC rules.

3.2 Labels

Labels are key/value tags you attach to any object (`metadata.labels`). Kubernetes itself does not care what they mean - *you* use them to group and select things. This is the glue holding Kubernetes together: Services find their Pods through labels, ReplicaSets count their Pods through labels, and so on.

labels example

```
apiVersion: v1
kind: Pod
metadata:
  name: shop-frontend-1
  labels:
    app: shop            # which application
    tier: frontend       # which layer
    env: prod            # which environment
    version: v2
spec:
  containers:
    - name: web
      image: nginx:1.27
```

Label rules: keys and values may contain letters, digits, `-`, `_`, `.` (max 63 characters per value). Prefixes like `app.kubernetes.io/name` are recommended for shared tooling. Use **annotations** (not labels) for non-identifying, free-form metadata such as build links or descriptions - you cannot select by annotation.

Terminal

```
kubectl get pods --show-labels
kubectl label pod shop-frontend-1 team=payments        # add a label
kubectl label pod shop-frontend-1 team-                 # remove it (trailing minus)
kubectl label pod shop-frontend-1 env=staging --overwrite
```

3.3 Selectors

A **selector** is a query over labels ("give me all Pods where ..."). There are two styles:

Equality-based selectors

Terminal

```
kubectl get pods -l app=shop                    # label equals
kubectl get pods -l env!=prod                   # label not equal
kubectl get pods -l app=shop,tier=frontend      # comma = AND
```

Set-based selectors (and the selector operators)

Terminal

```
kubectl get pods -l 'env in (prod,staging)'     # value is one of the set
kubectl get pods -l 'tier notin (backend)'      # value is not in the set
kubectl get pods -l 'team'                      # label key exists
kubectl get pods -l '!team'                     # label key does not exist
```

Inside YAML (Deployments, ReplicaSets, Jobs, DaemonSets, StatefulSets) the same ideas are written as **matchLabels** and **matchExpressions**:

selector syntax

```
selector:
  matchLabels:                    # simple equality (AND-ed together)
    app: shop
  matchExpressions:               # set-based rules (also AND-ed)
    - key: env
      operator: In
      values: [prod, staging]
    - key: tier
      operator: NotIn
      values: [backend]
    - key: team
      operator: Exists            # no 'values' allowed for Exists / DoesNotExist
    - key: deprecated
      operator: DoesNotExist
```

Operator	Meaning	Needs values?
`In`	Label value is in the given list	Yes
`NotIn`	Label value is not in the list (or label is absent)	Yes
`Exists`	Label key is present, any value	No
`DoesNotExist`	Label key is absent	No

Node affinity (Chapter 8) adds two more operators, `Gt` and `Lt`, for numeric comparisons.

Worked example

Suppose we have four Pods:

Pod	Labels
pod-1	app=shop, tier=frontend, env=prod
pod-2	app=shop, tier=backend, env=prod
pod-3	app=shop, tier=frontend, env=staging
pod-4	app=blog, tier=frontend, env=prod, team=web

Selector	Matches
`app=shop`	pod-1, pod-2, pod-3
`app=shop,env=prod`	pod-1, pod-2
`env in (staging)`	pod-3
`tier notin (backend)`	pod-1, pod-3, pod-4
`team`	pod-4
`!team`	pod-1, pod-2, pod-3

Where selectors are used

Service -> `spec.selector` (equality map only), ReplicaSet/Deployment -> `spec.selector.matchLabels`/`matchExpressions`, `kubectl -l`, NetworkPolicy, node/pod affinity. Mastering labels and selectors makes the next chapters easy.

4 Services

Stable networking for ever-changing Pods: ClusterIP, NodePort, LoadBalancer, ExternalName

Pods are mortal: they get created and destroyed and each new Pod gets a new IP. A **Service** gives a group of Pods one **stable virtual IP and DNS name** and load-balances traffic between the healthy ones. The Service finds its Pods using a **label selector**.

> **Real-life analogy**
> A Service is the **company's main phone number**. Employees (Pods) come and go, but customers always dial the same number and the switchboard forwards the call to whoever is available.

Under the hood, Kubernetes keeps a list of the ready Pod IPs behind each Service (called **EndpointSlices**), and `kube-proxy` on every node programs rules so traffic to the Service IP is spread across those Pods. Only Pods that pass their **readiness probe** receive traffic.

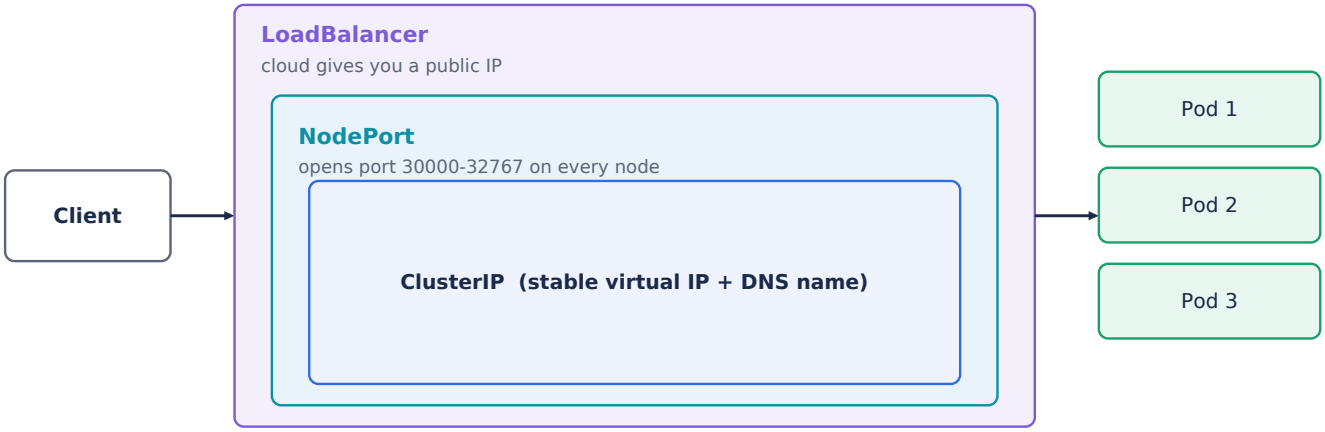


Figure 4.1 - Each Service type builds on the one inside it

4.1 ClusterIP (default)

Exposes the Service on an **internal IP reachable only inside the cluster**. Perfect for backend-to-backend traffic such as the web tier calling the API or a database.

clusterip.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  type: ClusterIP           # optional - this is the default
  selector:
    app: shop
    tier: backend           # send traffic to Pods with these labels
  ports:
    - name: http
      port: 80              # port of the Service
      targetPort: 3000      # port the container listens on
```

Test from inside the cluster

```
kubectl get svc backend                                 # shows the CLUSTER-IP
kubectl get endpoints backend                           # the Pod IPs behind it
kubectl run t --image=busybox:1.36 -it --rm -- wget -qO- http://backend
```

4.2 NodePort

Opens the **same port on every node** (default range **30000-32767**) and forwards it to the Service. You can then reach the app at `http://<any-node-ip>:<nodePort>`. Great for demos and local clusters; not elegant for production because you expose raw node ports.

nodeport.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080        # optional; Kubernetes picks one if omitted
```

4.3 LoadBalancer

Asks the **cloud provider** to create an external load balancer (AWS ELB/NLB, Azure LB, Google Cloud LB) with a public IP that sends traffic to the nodes. It includes a NodePort and ClusterIP automatically. On bare-metal clusters, install **MetalLB** to get the same behaviour.

loadbalancer.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web-lb
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

`kubectl get svc web-lb` shows `EXTERNAL-IP <pending>` for a minute or two, then the public address. Each LoadBalancer Service usually creates a separate (billable) cloud load balancer - for many web apps, one **Ingress** (Chapter 12) is cheaper.

4.4 ExternalName

Has **no selector and no proxying**. It just creates a DNS alias (CNAME) so Pods can use a friendly in-cluster name for something that lives outside, such as a managed database. If the external address changes later, you update the Service only.

externalname.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: orders-db
spec:
  type: ExternalName
  externalName: orders.abc123.ap-south-1.rds.amazonaws.com
```

Pods now connect to `orders-db` and DNS answers with the RDS hostname.

4.5 Comparison and bonus: Headless Service

Type	Reachable from	Typical use	Notes
ClusterIP	Inside cluster only	Internal microservice calls	Default type
NodePort	Outside, via `NodeIP:port`	Dev / test, simple exposure	Port 30000-32767
LoadBalancer	Outside, via cloud LB IP	Production exposure of one Service	Needs cloud / MetalLB; costs money
ExternalName	Inside cluster (DNS alias)	Point to external system	No Pods, no proxy

Headless Service (clusterIP: None)

Setting `clusterIP: None` gives no virtual IP; DNS returns the **individual Pod IPs** instead. It is used with **StatefulSets** (Chapter 7) when clients must talk to a specific Pod.

port vs targetPort vs nodePort

port = where the Service listens. **targetPort** = where the Pod's container listens. **nodePort** = the port opened on every node. Traffic flows `nodePort -> port -> targetPort`.

5 ReplicationController, ReplicaSet and Deployment

Keeping the right number of Pods alive, and updating them safely

If you create a bare Pod and it crashes, or its node dies, **nobody recreates it**. To get self-healing and scaling we use a **controller** that continuously compares the number of running Pods with the number you asked for.

> **Real-life analogy**
> Imagine a shift manager who must always have **3 cashiers** at the counter. If one leaves, the manager immediately calls in a replacement. That manager is a ReplicaSet. A **Deployment** is the regional boss who also plans *how* to swap in a new uniform (new version) without closing the shop.

5.1 ReplicationController (the original)

The oldest controller. It keeps a fixed number of Pod copies running. It only supports **equality-based** selectors and has been **superseded by ReplicaSet/Deployment**; you will mostly meet it in legacy manifests and exam questions.

rc.yaml

```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: web-rc
spec:
  replicas: 3
  selector:
    app: web               # plain key=value only
  template:                # the Pod blueprint
    metadata:
      labels:
        app: web           # must match the selector above
    spec:
      containers:
        - name: nginx
          image: nginx:1.26
```

5.2 ReplicaSet (the improved version)

Does the same job but with **set-based selectors** (`matchLabels` and `matchExpressions`, see Chapter 3). It is rarely created by hand: a Deployment creates and manages ReplicaSets for you.

replicaset.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.26
```

Watch the self-healing

```
kubectl apply -f replicaset.yaml
kubectl get rs                                  # DESIRED / CURRENT / READY
kubectl delete pod <one-of-the-pods>            # a replacement appears within seconds
kubectl scale rs web-rs --replicas=5            # scale up (or down)
```

ReplicaSet does not do rolling updates

If you edit the image in a ReplicaSet, **existing Pods are not touched**; only Pods created later use the new image. That limitation is exactly why Deployments exist.

5.3 Deployment (what you use every day)

A **Deployment** manages ReplicaSets and adds the features real applications need: **rolling updates, rollbacks, pause/resume and revision history**. Whenever you change the Pod template (for example the image tag), the Deployment creates a *new* ReplicaSet and gradually moves Pods from the old one to the new one. Old ReplicaSets are kept (scaled to 0) so you can roll back.

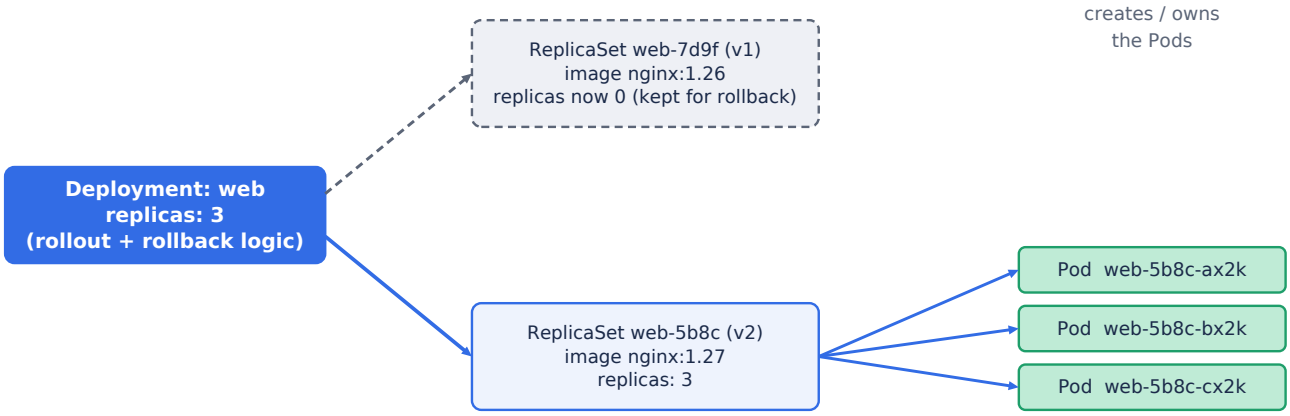


Figure 5.1 - Deployment -> ReplicaSet -> Pods. An update creates a second ReplicaSet

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  revisionHistoryLimit: 10        # how many old ReplicaSets to keep
  selector:
    matchLabels:
      app: web
  strategy:
    type: RollingUpdate           # default
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.26
          ports:
            - containerPort: 80
```

Daily Deployment commands

```
kubectl apply -f deployment.yaml
kubectl get deploy,rs,pods                        # see the whole chain
kubectl set image deployment/web nginx=nginx:1.27   # trigger a rolling update
kubectl rollout status deployment/web             # watch progress
kubectl rollout history deployment/web            # list revisions
kubectl rollout undo deployment/web               # roll back to previous revision
kubectl rollout undo deployment/web --to-revision=1
kubectl rollout restart deployment/web            # re-create Pods with same spec
kubectl scale deployment web --replicas=6
kubectl rollout pause deployment/web              # batch several edits, then 'resume'
```

Pod names explained

A Pod named `web-5b8c9d7f6-x2k4q` = Deployment name (`web`) + ReplicaSet template hash (`5b8c9d7f6`) + random suffix. Seeing the hash change tells you a new rollout happened.

Feature	ReplicationController	ReplicaSet	Deployment
Keeps N Pods running	Yes	Yes	Yes (via ReplicaSet)
Selector type	Equality only	Equality + set-based	Equality + set-based
Rolling update	No (manual, old `rolling-update` command removed)	No	**Yes**
Rollback / history	No	No	**Yes**
Recommended today	No (legacy)	Rarely, indirectly	**Yes**, for stateless apps

6 Deployment Strategies

RollingUpdate, Recreate, Blue-Green and Canary

A **deployment strategy** is the plan for replacing the old version of your app with the new one. Kubernetes has **two built-in strategies** (`RollingUpdate` and `Recreate`). **Blue-Green** and **Canary** are *patterns* you build yourself with labels, Services and multiple Deployments (or with tools such as Argo Rollouts, Gateway API or a service mesh).

6.1 RollingUpdate (default)

Replaces Pods gradually, a few at a time, so the app stays available. Two knobs control the pace:

Setting	Meaning	Default
`maxSurge`	How many **extra** Pods above `replicas` may exist during the update (number or %)	25%
`maxUnavailable`	How many Pods may be **unavailable** during the update (number or %)	25%

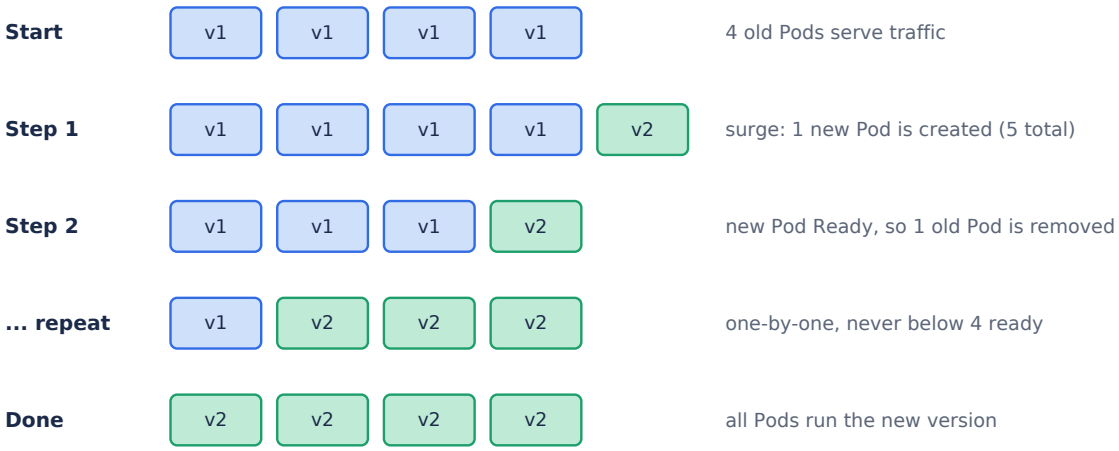


Figure 6.1 - Rolling update with replicas=4, maxSurge=1, maxUnavailable=0

rolling update settings

```
spec:
  replicas: 4
  minReadySeconds: 5              # new Pod must stay Ready 5 s before it counts
  progressDeadlineSeconds: 300    # mark rollout failed if stuck for 5 min
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1                 # at most 5 Pods at any time
      maxUnavailable: 0           # always keep all 4 old/new Pods serving
```

- **Pros:** zero downtime, simple, built in, easy rollback with `kubectl rollout undo`.
- **Cons:** for a short time **both versions run together**, so v1 and v2 must be compatible (database schema, APIs). Rollback is gradual, not instant.
- **Key requirement:** a correct **readiness probe** (Chapter 11). Without it Kubernetes cannot tell when a new Pod is truly ready.

6.2 Recreate

Stops **all** old Pods first, then starts the new ones. There is a period of **downtime**, but the two versions never overlap.

recreate

```
spec:
  strategy:
    type: Recreate
```

- **Use when:** the versions cannot run side by side (breaking schema change), the app holds an exclusive lock or a single-attach (ReadWriteOnce) volume, or short downtime is acceptable (dev/test, batch).
- **Pros:** simplest, clean cut-over, lowest resource use. **Cons:** downtime.

6.3 Blue-Green

Run **two full environments**: *blue* (current, live) and *green* (new version). Test green while users still use blue, then **flip the Service selector** to green in one instant. To roll back, flip it back.

blue-green.yaml

```
# --- blue (live) ---
apiVersion: apps/v1
kind: Deployment
metadata: { name: myapp-blue }
spec:
  replicas: 3
  selector:
    matchLabels: { app: myapp, version: blue }
  template:
    metadata:
      labels: { app: myapp, version: blue }
    spec:
      containers:
        - name: app
          image: myorg/myapp:1.0
---
# --- green (new) : identical except name, label 'version: green' and image ---
#   metadata.name: myapp-green | labels.version: green | image: myorg/myapp:2.0
---
apiVersion: v1
kind: Service
metadata: { name: myapp }
spec:
  selector:
    app: myapp
    version: blue                # <-- this line decides who gets traffic
  ports:
    - port: 80
      targetPort: 8080
```

Switch traffic

```
# Test green first:  kubectl port-forward deploy/myapp-green 8081:8080
# Cut over:
kubectl patch service myapp -p '{"spec":{"selector":{"app":"myapp","version":"green"}}}'
# Roll back instantly:
kubectl patch service myapp -p '{"spec":{"selector":{"app":"myapp","version":"blue"}}}'
# When confident, remove the old one:  kubectl delete deployment myapp-blue
```

- **Pros:** instant switch and instant rollback; full testing in production-like conditions; no mixed versions for users.
- **Cons:** needs **double resources** while both run; databases and in-flight sessions need extra care.

6.4 Canary

Send a **small slice of real traffic** (say 10%) to the new version first. If metrics look healthy, increase the share; if not, remove the canary and only a few users were affected.

canary.yaml (Service)

```
# Stable: 9 replicas, labels app=myapp, track=stable     (image myapp:1.0)
# Canary: 1 replica,  labels app=myapp, track=canary     (image myapp:2.0)
apiVersion: v1
kind: Service
metadata: { name: myapp }
spec:
  selector:
    app: myapp                    # NO 'track' label -> matches BOTH Deployments
  ports:
    - port: 80
      targetPort: 8080
```

Adjust the split

```
kubectl scale deployment myapp-canary --replicas=3       # grow the canary...
kubectl scale deployment myapp-stable --replicas=7       # ...shrink stable (30% canary)
# Happy? update stable to 2.0 (rolling update) and delete the canary Deployment.
# Not happy? kubectl delete deployment myapp-canary
```

Plain Kubernetes splits traffic **by Pod count** (1 of 10 Pods = about 10%). For exact percentages (1%, 5%) or header-based routing, use **Gateway API** weighted routes, a **service mesh** (Istio, Linkerd) or **Argo Rollouts / Flagger**.

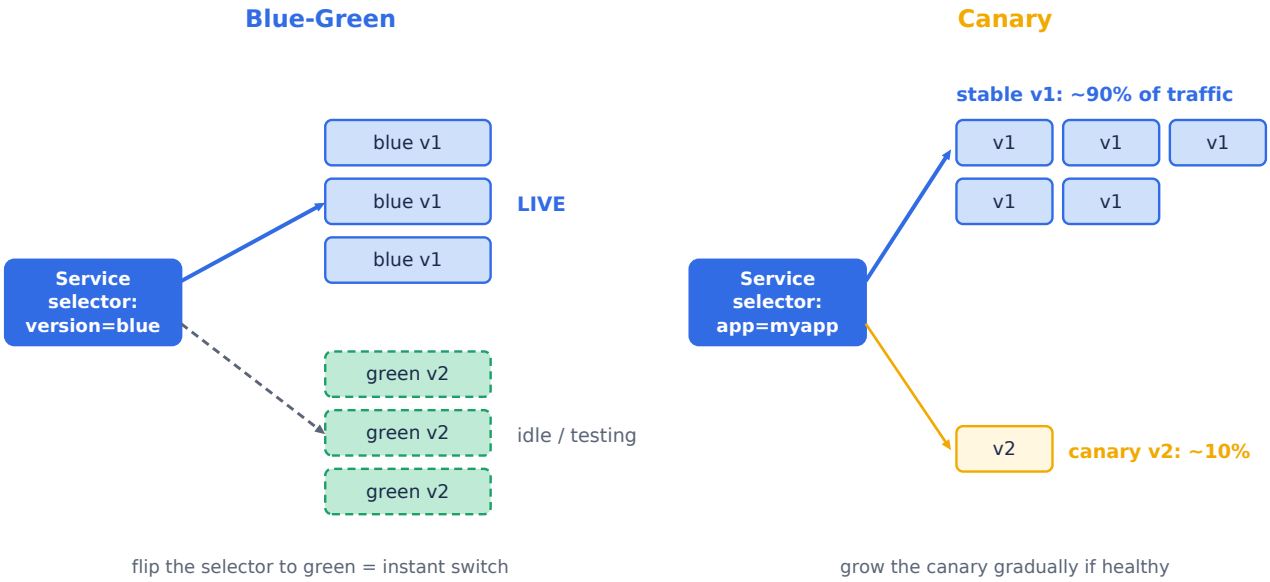


Figure 6.2 - Blue-Green flips a Service selector; Canary mixes Pods behind one selector

	RollingUpdate	Recreate	Blue-Green	Canary
Downtime	None	Yes	None	None
Extra resources	Small (surge)	None	**Double**	Small
Rollback speed	Gradual	Redeploy	**Instant**	Fast (delete canary)
Versions at once	Yes (brief)	No	No (switch)	Yes (long)
Risk exposure	Medium	High (outage)	Low	**Lowest**
Built in?	Yes	Yes	Pattern	Pattern

7 StatefulSet and DaemonSet

Workloads that need identity, or need to run everywhere

7.1 StatefulSet: for apps that remember

Pods from a Deployment are **interchangeable**: random names, shared storage, any start order. Databases and clustered systems (MySQL, PostgreSQL, MongoDB, Kafka, ZooKeeper, Elasticsearch) need more. A **StatefulSet** gives each Pod:

- **Stable, predictable names**: `web-0`, `web-1`, `web-2` (not random).
- **Stable network identity** via a **headless Service**: `web-0.web.default.svc.cluster.local`.
- **Its own persistent disk** created from `volumeClaimTemplates`; the disk reattaches if the Pod is rescheduled.
- **Ordered start and stop**: 0, then 1, then 2 on scale-up (each must be Ready first); reverse order on scale-down.

Real-life analogy

A Deployment is a **pool of taxis**: any cab will do. A StatefulSet is your **named doctor team**: Dr. Rao (web-0) keeps *your* patient files (disk). If Dr. Rao is replaced mid-day, the new doctor gets the same name tag and the same filing cabinet.

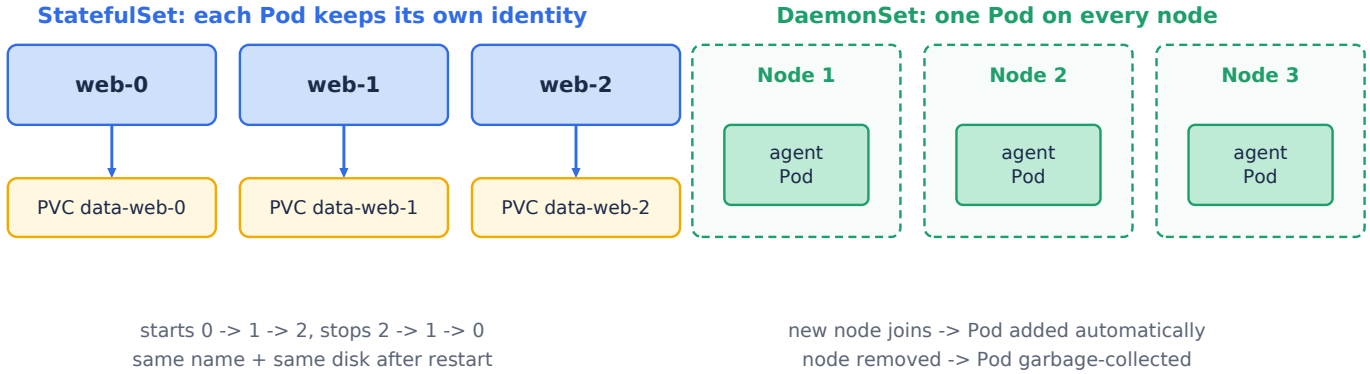


Figure 7.1 - StatefulSet (left) and DaemonSet (right)

statefulset.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web                     # the headless Service
spec:
  clusterIP: None               # headless: DNS returns each Pod's own IP
  selector:
    app: web
  ports:
    - port: 80
      name: web
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: web              # must reference the headless Service
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
          volumeMounts:
            - name: data
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:         # one PVC per Pod: data-web-0, data-web-1, ...
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi        # uses the cluster's default StorageClass
```

Prove the identity

```
kubectl get pods -w                     # watch web-0, web-1, web-2 appear in order
kubectl get pvc                         # data-web-0, data-web-1, data-web-2
kubectl exec web-0 -- sh -c 'echo hello > /usr/share/nginx/html/index.html'
kubectl delete pod web-0                # comes back as web-0 with the SAME disk
kubectl exec web-0 -- cat /usr/share/nginx/html/index.html    # still prints: hello
```

	Deployment	StatefulSet
Pod names	Random (`web-5b8c-x2k`)	Ordered (`web-0`, `web-1`)
Storage	Usually shared or none	Separate PVC per Pod
Start / stop order	All at once	Sequential (or `Parallel` if configured)
Network identity	Changes on replacement	Stable DNS per Pod
Best for	Stateless web apps / APIs	Databases, queues, clustered systems

Watch out

Deleting or scaling down a StatefulSet does **not delete its PVCs** - this protects your data, but remember to clean them up when you truly do not need them. In production, prefer a well-tested **operator** (for Postgres, MySQL, Kafka...) over hand-written StatefulSets.

7.2 DaemonSet: one Pod per node

A **DaemonSet** makes sure **every node** (or every node matching a rule) runs exactly one copy of a Pod. When a node joins the cluster, the Pod is added automatically; when a node leaves, the Pod is removed.

Typical DaemonSet workload	Examples
Log collection	Fluent Bit, Fluentd, Filebeat
Node monitoring	Prometheus node-exporter, Datadog agent
Networking	CNI plugins (Calico, Cilium), `kube-proxy`
Storage / security agents	CSI node drivers, runtime security scanners

daemonset.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
spec:
  selector:
    matchLabels:
      app: node-agent
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1          # update one node at a time
  template:
    metadata:
      labels:
        app: node-agent
    spec:
      # nodeSelector:            # uncomment to run only on some nodes
      #   disktype: ssd
      tolerations:               # also run on control plane nodes
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: agent
          image: busybox:1.36
          env:
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName    # tell the Pod which node it is on
          command: ["sh", "-c", "while true; do echo agent alive on $NODE_NAME; sleep 30; done"]
```

Terminal

```
kubectl get ds                              # DESIRED = number of eligible nodes
kubectl get pods -o wide -l app=node-agent   # one Pod on each node
```

8 Node Affinity, Taints and Tolerations

Telling the scheduler where Pods should (and should not) run

By default the scheduler places Pods on any node with enough free resources. Often that is not what you want: machine-learning Pods need GPU nodes, a database needs SSD nodes, or a node is reserved for one team. Kubernetes gives you two complementary tools:

Tool	Defined on	Direction	Think of it as
Node affinity (and `nodeSelector`)	The **Pod**	Pod is **attracted** to nodes	"I want a window seat"
Taints	The **node**	Node **repels** Pods	A "Staff only" sign on a door
Tolerations	The **Pod**	Lets a Pod ignore a taint	The staff badge that lets you in

8.1 nodeSelector (the simple way)

Terminal

```
kubectl label nodes worker-1 disktype=ssd          # first, label a node
kubectl get nodes --show-labels
```

pod snippet

```
spec:
  nodeSelector:
    disktype: ssd                # Pod runs only on nodes with this label
  containers:
    - name: db
      image: postgres:16
```

8.2 Node affinity (the flexible way)

Node affinity does the same job but with richer rules: operators (`In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt`, `Lt`), and **soft preferences** as well as hard requirements.

Type	Meaning
`requiredDuringSchedulingIgnoredDuringExecution`	**Hard rule.** Pod will not schedule unless a node matches (stays Pending otherwise).
`preferredDuringSchedulingIgnoredDuringExecution`	**Soft rule.** Scheduler tries to match; uses `weight` 1-100 for priority; otherwise places the Pod anywhere.

The long suffix `IgnoredDuringExecution` means: if a node's labels change *after* the Pod is running, the Pod is **not evicted**.

node-affinity.yaml

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:     # MUST
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: ["ap-south-1a", "ap-south-1b"]
      preferredDuringSchedulingIgnoredDuringExecution:    # NICE TO HAVE
        - weight: 80
          preference:
            matchExpressions:
              - key: disktype
                operator: In
                values: ["ssd"]
  containers:
    - name: app
      image: nginx:1.27
```

8.3 Taints and tolerations

A **taint** is a mark on a node: "*do not place Pods here unless they tolerate this*". It has the form `key=value:effect`.

Effect	What it does
`NoSchedule`	New Pods without a matching toleration are **not scheduled** here. Running Pods stay.
`PreferNoSchedule`	Soft version: scheduler **tries to avoid** the node but may use it.
`NoExecute`	New Pods are blocked **and existing Pods without a toleration are evicted**.

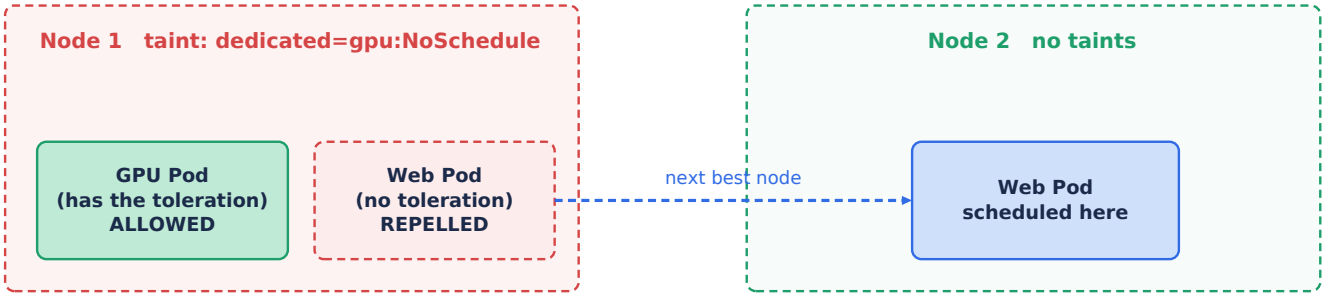


Figure 8.1 - Only Pods that tolerate the taint may enter the tainted node

Terminal

```
kubectl taint nodes worker-2 dedicated=gpu:NoSchedule       # add a taint
kubectl describe node worker-2 | grep -i taints             # view it
kubectl taint nodes worker-2 dedicated=gpu:NoSchedule-      # remove it (note the minus)
```

toleration.yaml

```
spec:
  tolerations:
    - key: "dedicated"
      operator: "Equal"           # key AND value must match
      value: "gpu"
      effect: "NoSchedule"
    - key: "node.kubernetes.io/unreachable"
      operator: "Exists"          # only the key needs to exist
      effect: "NoExecute"
      tolerationSeconds: 300      # stay 5 min on a failing node before eviction
  containers:
    - name: trainer
      image: myorg/trainer:1.0
```

> **Good to know**
> Control plane nodes carry the taint `node-role.kubernetes.io/control-plane:NoSchedule`, which is why ordinary Pods do not land there. Kubernetes also adds a default 300-second toleration for `not-ready` and `unreachable` nodes to every Pod.

8.4 Putting it together: a dedicated GPU pool

A toleration only **allows** a Pod onto a tainted node; it does **not force** it there. To get a true dedicated pool, combine both:

1. **Taint** the GPU nodes -> normal Pods stay away.
2. **Label** the GPU nodes (e.g. `hardware=gpu`).
3. Give GPU Pods a **toleration** (allowed in) **and** a **node affinity / nodeSelector** (must go there).

8.5 Bonus: Pod affinity and anti-affinity

Node affinity looks at *node* labels. **Pod anti-affinity** looks at *other Pods*: it is the usual way to spread replicas across nodes so one node failure cannot take down every copy. (`topologySpreadConstraints` is a newer, more flexible alternative.)

anti-affinity.yaml

```
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app: web                        # avoid nodes already running 'web' Pods
            topologyKey: kubernetes.io/hostname   # 'same node' = same hostname
```

9 ConfigMap and Secret

Keeping configuration and credentials out of your container image

A good container image is **the same everywhere** (dev, test, prod). What changes is configuration: database URLs, feature flags, passwords. Kubernetes keeps these in separate objects and injects them into Pods when they start.

Real-life analogy
The container image is a **printed manual**; a ConfigMap is a **sticky note** you attach for this particular office ("Wi-Fi network: Office-5G"). A Secret is a **locked envelope** for sensitive items such as the Wi-Fi password.

	ConfigMap	Secret
Holds	Non-sensitive settings	Passwords, tokens, keys, certificates
Stored as	Plain text	Base64-encoded (**not encrypted by default**)
Size limit	1 MiB	1 MiB
Consumed as	Env vars, files, command args	Env vars or files (files preferred)

9.1 ConfigMap

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "production"          # simple key/value
  LOG_LEVEL: "info"
  app.properties: |               # a whole file as one value
    cache.size=100
    feature.beta=true
```

Imperative creation

```
kubectl create configmap app-config --from-literal=APP_MODE=production --from-literal=LOG_LEVEL=info
kubectl create configmap app-files --from-file=app.properties      # from a file
kubectl get configmap app-config -o yaml
kubectl describe configmap app-config
```

9.2 Secret

Secrets look similar but are handled with more care (kept in memory on nodes, can be encrypted at rest, can be restricted with RBAC). Values in `data:` must be **base64**; use `stringData:` to write plain text and let Kubernetes encode it.

secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:                           # base64 encoded
  username: YWRtaW4=            # admin
  password: cGFzc3dvcmQ=        # password   (demo only!)
# stringData:                   # alternative: plain text, encoded for you
#   username: admin
```

Terminal

```
echo -n 'admin' | base64                                  # YWRtaW4=
kubectl create secret generic db-secret --from-literal=username=admin --from-literal=password='S3cr3t!'
kubectl get secret db-secret -o jsonpath='{.data.password}' | base64 -d     # decode
kubectl create secret docker-registry regcred --docker-server=<registry> \
    --docker-username=<user> --docker-password=<token>      # pull images from a private registry
kubectl create secret tls shop-tls --cert=tls.crt --key=tls.key    # TLS certificate for Ingress
```

Secret type	Used for
`Opaque`	Generic key/value data (default)
`kubernetes.io/dockerconfigjson`	Registry credentials (`imagePullSecrets`)
`kubernetes.io/tls`	TLS certificate + private key (Ingress, webhooks)
`kubernetes.io/basic-auth`, `ssh-auth`	Basic-auth or SSH credentials

Base64 is encoding, not encryption

Anyone who can read the Secret can decode it in one command. Protect it with: **RBAC** (limit who can `get secrets`), **encryption at rest** for etcd, never committing Secret YAML to Git, and for serious setups an external manager (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) synced in via the External Secrets Operator or the Secrets Store CSI driver.

9.3 Using them in a Pod

Three ways to consume the data:

- **One variable** with `valueFrom` (`configMapKeyRef` / `secretKeyRef`).
- **All keys as variables** with `envFrom`.
- **As files** by mounting a volume: each key becomes a file. Mounted files update automatically (after a short delay) when the ConfigMap/Secret changes; **environment variables do not**, they need a Pod restart.

config-demo.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: config-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "env | grep -E 'APP_|LOG_|DB_'; cat /etc/config/app.properties; sleep 3600"]
      envFrom:
        - configMapRef:
            name: app-config              # APP_MODE and LOG_LEVEL become env vars
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: username
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: password
      volumeMounts:
        - name: config-vol
          mountPath: /etc/config          # file: /etc/config/app.properties
        - name: secret-vol
          mountPath: /etc/secret
          readOnly: true
  volumes:
    - name: config-vol
      configMap:
        name: app-config
    - name: secret-vol
      secret:
        secretName: db-secret
```

Tip

Add `immutable: true` to a ConfigMap or Secret you never intend to change: it protects against accidental edits and lowers load on the API server. To roll out a config change safely, give the object a new name (e.g. `app-config-v2`) and update the Deployment to point at it.

10 PV, PVC, StorageClass and Provisioning

Giving Pods storage that outlives them

A container's own filesystem is **temporary**: when the container restarts, the data is gone. For data that must survive (databases, uploads) we use **Persistent Volumes**. Kubernetes separates *who provides* storage from *who uses* it:

> **Real-life analogy**
> Think of renting a **storage unit**. The **PV** is the actual unit in the warehouse. The **PVC** is your rental request ("I need 10 GB, single user"). The **StorageClass** is the price list of unit types (fast SSD, cheap HDD) that lets the warehouse build a new unit automatically when you ask.

Object	Who creates it	What it is
PersistentVolume (PV)	Admin, or automatically by a StorageClass	A piece of real storage registered in the cluster (cluster-wide, not namespaced)
PersistentVolumeClaim (PVC)	Developer	A request for storage of a certain size and access mode (namespaced). Pods mount the PVC.
StorageClass (SC)	Admin	A template describing *how* to create storage (provisioner, disk type, reclaim policy)

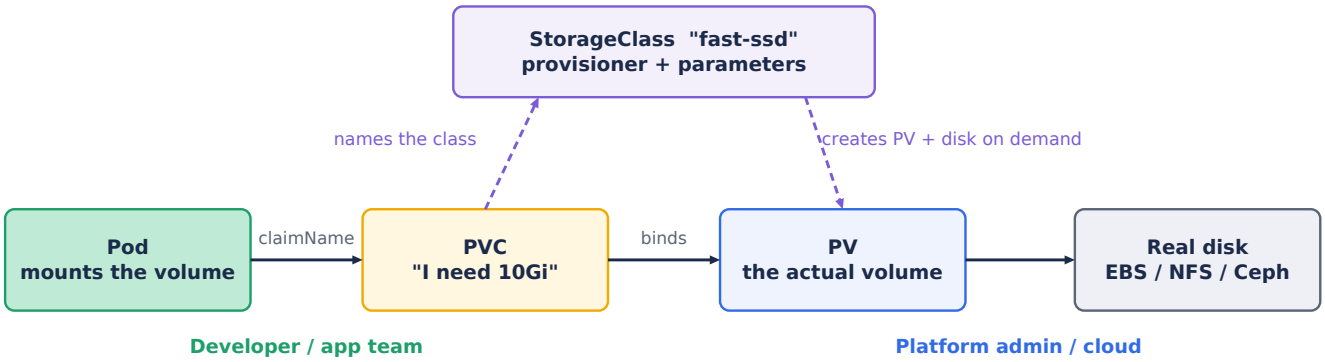


Figure 10.1 - Pod -> PVC -> PV -> disk, with a StorageClass creating volumes on demand

10.1 Other volume types in one minute

Volume	Lifetime	Use
`emptyDir`	Lives and dies with the Pod	Scratch space, sharing files between containers
`hostPath`	Node's disk directory	Single-node demos; avoid in production (ties Pod to a node, security risk)
PVC-backed	Independent of the Pod	Databases and anything that must persist

10.2 Static provisioning (admin creates the PV)

static-pv-pvc.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  capacity:
    storage: 5Gi
  accessModes: ["ReadWriteOnce"]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:                       # fine for learning on minikube/kind
    path: /mnt/data
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: manual        # must match the PV to bind
  resources:
    requests:
      storage: 2Gi                # needs a PV with at least 2Gi
```

pod-with-pvc.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: pv-pod
spec:
  containers:
    - name: web
      image: nginx:1.27
      volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: pvc-demo       # Pod mounts the CLAIM, never the PV directly
```

Terminal

```
kubectl get pv,pvc                # STATUS should show Bound
kubectl describe pvc pvc-demo     # events show why it is Pending, if it is
```

10.3 Dynamic provisioning with a StorageClass

Creating PVs by hand does not scale. With a **StorageClass**, the developer just creates a PVC and the cluster's storage driver (a **CSI driver**) creates the disk and the PV automatically.

storageclass.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  # annotations:
  #   storageclass.kubernetes.io/is-default-class: "true"   # make it the cluster default
provisioner: ebs.csi.aws.com      # the CSI driver (example: AWS EBS)
parameters:
  type: gp3                       # driver-specific options
reclaimPolicy: Delete             # delete the disk when the PVC is deleted
volumeBindingMode: WaitForFirstConsumer   # create the disk in the same zone as the Pod
allowVolumeExpansion: true        # let PVCs be resized later
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: fast-ssd      # <- triggers dynamic provisioning
  resources:
    requests:
      storage: 10Gi
```

1. Developer applies the **PVC** naming `fast-ssd`.
2. The provisioner behind that StorageClass **creates a real disk** (e.g. a 10 GiB EBS volume).
3. Kubernetes **creates a PV** for it and **binds** it to the PVC.
4. With `WaitForFirstConsumer`, steps 2-3 wait until a Pod using the PVC is scheduled, so the disk is created in the right zone.

10.4 Access modes, reclaim policies and phases

Access mode	Short	Meaning
ReadWriteOnce	RWO	Read/write by **one node** at a time (most block disks)
ReadOnlyMany	ROX	Read-only by many nodes
ReadWriteMany	RWX	Read/write by many nodes (needs NFS, CephFS, EFS, Azure Files...)
ReadWriteOncePod	RWOP	Read/write by a **single Pod** only

Reclaim policy	When the PVC is deleted...
`Retain`	PV and data are **kept** (status `Released`); admin cleans up manually. Safest for important data.
`Delete`	PV **and the underlying disk are deleted**. Default for dynamically created volumes.

PV status: **Available** (free) -> **Bound** (claimed) -> **Released** (claim deleted, not yet reclaimed) / **Failed**. A PVC stays **Pending** if no PV matches its size, access mode and class.

> **Tip**
> Expand a volume by editing the PVC's `requests.storage` to a larger value (only if the StorageClass has `allowVolumeExpansion: true`). Shrinking is not supported.

11 Readiness, Liveness and Startup Probes

Teaching Kubernetes to tell whether your app is really healthy

"Container is running" does not mean "app is working". An app can be deadlocked, still loading data, or temporarily unable to reach its database. **Probes** are health checks the kubelet runs against your container to decide what to do.

Real-life analogy

A **shop**: the **startup probe** asks "has the owner finished opening up?" (give a slow starter time). The **readiness probe** is the **OPEN / CLOSED sign** (closed during a stock-take; customers are sent elsewhere but the shop is not demolished). The **liveness probe** is a **pulse check**: no pulse means the owner is replaced.

Probe	Question it answers	If it fails	Typical use
Startup	Has the app finished starting?	After `failureThreshold` failures the container is **restarted**. Until it succeeds, liveness and readiness are **disabled**.	Slow-booting apps (Java, big caches)
Liveness	Is the app alive or stuck?	Container is **killed and restarted** (per restartPolicy)	Detect deadlocks, hung processes
Readiness	Can it serve traffic right now?	Pod is **removed from Service endpoints** (no restart); added back when it passes	Warm-up, temporary overload, dependency outage

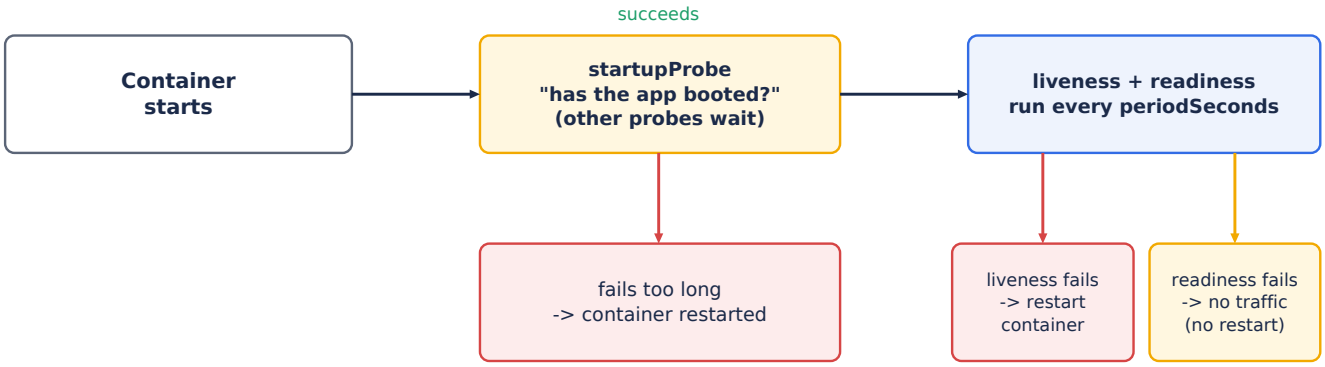


Figure 11.1 - How the three probes work together

11.1 Probe mechanisms

Mechanism	Success means	Example use
`httpGet`	HTTP status 200-399	Web apps and APIs with a `/healthz` endpoint
`tcpSocket`	Port accepts a TCP connection	Databases, caches, non-HTTP services
`exec`	Command exits with code 0	Check a file or run a CLI health command
`grpc`	gRPC health check returns SERVING	gRPC services

11.2 Tuning parameters

Field	Meaning	Default
initialDelaySeconds	Wait this long after the container starts before the first probe	0
periodSeconds	How often to probe	10
timeoutSeconds	How long to wait for a response	1
successThreshold	Consecutive successes needed to be considered healthy again (must be 1 for liveness/startup)	1
failureThreshold	Consecutive failures before taking action	3

11.3 Complete example

probes.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      containers:
        - name: api
          image: myorg/api:1.0
          ports:
            - containerPort: 8080
          startupProbe:                 # allows up to 30 x 10 s = 300 s to boot
            httpGet: { path: /healthz, port: 8080 }
            periodSeconds: 10
            failureThreshold: 30
          livenessProbe:                # "am I stuck?"  restart if 3 checks fail
            httpGet: { path: /healthz, port: 8080 }
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          readinessProbe:               # "can I serve now?"  stop traffic if it fails
            httpGet: { path: /ready, port: 8080 }
            periodSeconds: 5
            failureThreshold: 3
```

other mechanisms

```
livenessProbe:                  # exec: healthy while the command exits 0
  exec:
    command: ["cat", "/tmp/healthy"]
readinessProbe:                 # tcpSocket: healthy while the port accepts connections
  tcpSocket:
    port: 5432
```

Debugging probes

```
kubectl describe pod <pod>        # Events show: "Liveness probe failed: HTTP probe failed with statuscode: 500"
kubectl get pods                  # READY 0/1 = readiness failing; RESTARTS rising = liveness failing
kubectl get endpoints api         # Pods failing readiness are missing from this list
```

11.4 Probes, Services and rolling updates

Readiness is the link between probes and the rest of this guide:

- A **Service** (Chapter 4) only sends traffic to **Ready** Pods; a failing readiness probe removes the Pod from the endpoint list within seconds.
- A **rolling update** (Chapter 6) waits for each new Pod to become Ready before removing an old one. A good readiness probe is what makes the update zero-downtime, and a broken new version **stops the rollout** instead of replacing healthy Pods.
- A failing **liveness** probe restarts the container, which shows up as a rising `RESTARTS` count.

Situation	Rule-of-thumb settings (adjust for your app)
Fast web API	readiness `periodSeconds: 5`, `failureThreshold: 3`; liveness `periodSeconds: 10`
Slow-starting app (e.g. Java)	startup probe with `failureThreshold` x `periodSeconds` greater than the worst-case boot time; keep liveness tight afterwards
Background worker with no port	`exec` liveness (e.g. heartbeat file is recent); readiness usually not needed

Common mistakes

1) A liveness probe that checks **external dependencies** (database, other services): when the DB blips, *every* Pod restarts and makes it worse. Keep liveness about the app's own health; put dependency checks in readiness. 2) Too-aggressive timings (`timeoutSeconds: 1` on a slow endpoint) causing needless restarts. 3) No readiness probe, so rolling updates send traffic to Pods that are not ready yet.

12 Ingress, Ingress Controller and Ingress Rules

One entry point for many web applications

A `LoadBalancer` Service gives one external IP per Service, which gets expensive and messy with many apps. **Ingress** lets you expose many Services through **one entry point**, routing by **hostname and URL path** at layer 7 (HTTP/HTTPS), and terminate **TLS** (HTTPS) in one place.

Real-life analogy
Ingress is an **office building's directory board** ("Accounts: Floor 2, HR: Floor 3"). The **Ingress Controller** is the **receptionist** who actually reads the board and walks each visitor to the right floor. A directory board with no receptionist does nothing.

12.1 Two separate pieces

Piece	What it is
Ingress resource	A Kubernetes object containing **routing rules** (host, path -> Service). Just data; by itself it does nothing.
Ingress Controller	A running program (Pods + usually a LoadBalancer Service) that **watches Ingress objects and configures a reverse proxy** accordingly. You must install one.
IngressClass	Names which controller should handle which Ingress (`ingressClassName`). Lets several controllers coexist.

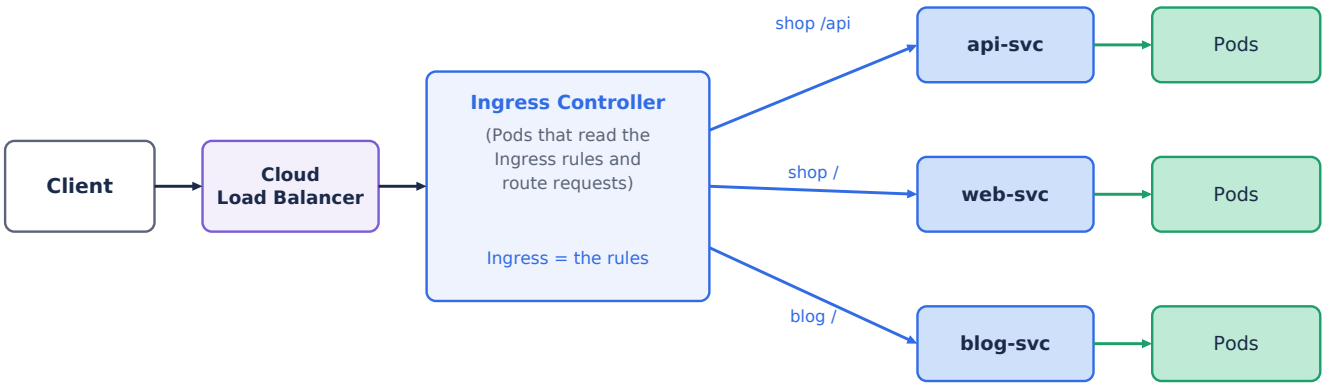


Figure 12.1 - Request flow: Client -> Load Balancer -> Ingress Controller -> Service -> Pods

Controller choice: check the project's status
The community **Ingress NGINX** controller was announced for retirement, with best-effort maintenance ending in **March 2026** and no further security fixes afterwards; existing installs keep running. For new clusters pick an actively maintained controller (Traefik, HAProxy, Envoy-based Contour/Emissary, F5 NGINX Ingress Controller, Kong, your cloud's native controller) or move to **Gateway API** (see 12.4). Examples below use `ingressClassName: nginx` only as a placeholder: put the class name of **your** controller.

12.2 Ingress rules

Each rule matches a **host** (optional) and one or more **paths**, then forwards to a **backend Service and port**.

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop-ingress
spec:
  ingressClassName: nginx           # which controller handles this (use your controller's class)
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls          # Secret of type kubernetes.io/tls
  rules:
    - host: shop.example.com        # host-based rule
      http:
        paths:
          - path: /api              # path-based rules
            pathType: Prefix
            backend:
              service:
                name: api-svc
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-svc
                port:
                  number: 80
    - host: blog.example.com        # a second site on the same entry point
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: blog-svc
                port:
                  number: 80
```

pathType	Behaviour	Example: rule /api
`Exact`	Matches only that exact path	`/api` yes, `/api/users` no
`Prefix`	Matches the path and everything below it, by path segments	`/api`, `/api/users` yes; `/apis` no
`ImplementationSpecific`	Up to the controller (may allow regex)	Depends on the controller

- Longest, most specific path wins; `/` acts as the catch-all.
- `defaultBackend` handles requests matching no rule.
- **Annotations** (e.g. rewrite rules, timeouts, rate limits) are controller-specific: read your controller's documentation.

Terminal

```
kubectl get ingress                       # shows ADDRESS once the controller picks it up
kubectl describe ingress shop-ingress     # rules, backends, events
# Test without DNS, send the Host header yourself:
curl -H "Host: shop.example.com" http://<ingress-address>/api
```

12.3 Installing a controller (concept)

1. Install a controller, normally with **Helm** or the vendor's manifests, into its own namespace.
2. It exposes itself with a `LoadBalancer` Service (cloud) or `NodePort`/MetalLB (bare metal): this is the single public IP.

3. Point your DNS records (`shop.example.com`, `blog.example.com`) at that address.
4. Create Ingress objects; the controller reloads its proxy config automatically.

12.4 Looking ahead: Gateway API

Gateway API is the newer, more expressive successor to Ingress, now the direction recommended by the Kubernetes project. It splits responsibilities: platform teams manage **Gateways** (listeners, TLS), app teams manage **HTTPRoutes** (rules), and features like weighted traffic splitting (handy for canaries) are built in. The same shop route would look like this:

httproute.yaml

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shop-route
spec:
  parentRefs:
    - name: my-gateway               # the Gateway managed by the platform team
  hostnames: ["shop.example.com"]
  rules:
    - matches:
        - path: { type: PathPrefix, value: /api }
      backendRefs:
        - name: api-svc
          port: 80
          weight: 100                # e.g. split 90/10 between two backends for a canary
```

13 HPA, VPA and Cluster Autoscaler

Letting the cluster grow and shrink with demand

Traffic is rarely constant. Kubernetes offers three autoscalers that work at **three different levels**:

Real-life analogy

A **supermarket**. **HPA** opens more **checkout counters** when queues grow (more Pods). **VPA** gives each cashier a **bigger desk** if their job needs it (more CPU/memory per Pod). **Cluster Autoscaler builds extra floor space** when the store is full (more nodes).

Autoscaler	Scales	Direction	Built in?
HPA Horizontal Pod Autoscaler	Number of Pods	Out / in	Yes (needs metrics-server)
VPA Vertical Pod Autoscaler	CPU/memory requests of each Pod	Up / down	No, installed separately (add-on)
Cluster Autoscaler	Number of nodes	Out / in	No, add-on for your cloud / node groups

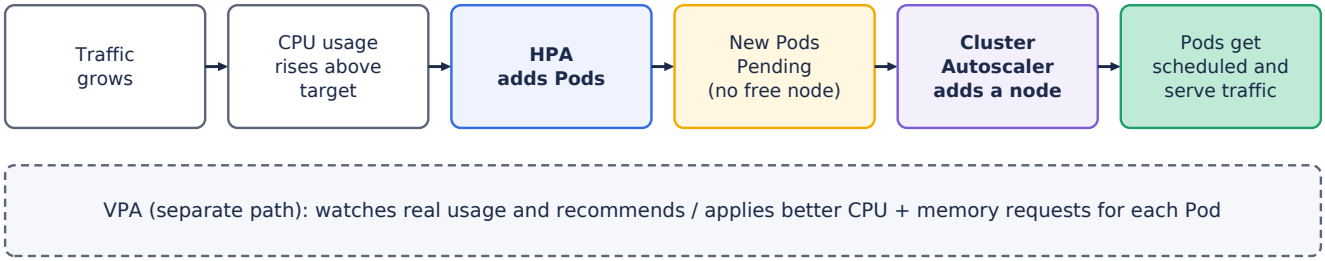


Figure 13.1 - HPA and Cluster Autoscaler working in sequence; VPA tunes Pod size

13.1 Prerequisites

- **metrics-server** installed (`kubectl top pods` must work); it supplies CPU/memory metrics.
- Containers must define **resource requests**. HPA measures utilization as a **percentage of the request**; with no request it cannot calculate it.

13.2 Horizontal Pod Autoscaler (HPA)

The HPA checks metrics (every ~15 s) and adjusts `replicas` of a Deployment/StatefulSet between your minimum and maximum. The core formula:

How HPA decides

```
desiredReplicas = ceil( currentReplicas  x  currentMetricValue / targetMetricValue )

Example: 4 Pods are averaging 80% CPU, target is 50%
         ceil( 4 x 80 / 50 ) = ceil(6.4) = 7 Pods
```

hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:                # what to scale
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50  # keep average CPU near 50% of the requests
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300   # wait 5 min of low load before shrinking (avoids flapping)
```

Terminal

```
kubectl autoscale deployment web --cpu-percent=50 --min=2 --max=10    # one-line alternative
kubectl get hpa -w                                                    # TARGETS shows current/target
kubectl top pods                                                      # live CPU / memory
# generate load to watch it scale:
kubectl run load --image=busybox:1.36 -it --rm -- sh -c 'while true; do wget -qO- http://web; done'
```

HPA can also scale on **memory**, **custom metrics** (requests per second, queue length) and **external metrics**, using a metrics adapter (for example Prometheus Adapter) or event-driven tools like **KEDA**.

13.3 Vertical Pod Autoscaler (VPA)

Guessing the right CPU/memory requests is hard. VPA watches real usage and **recommends** (or applies) better requests. It runs as three components: **Recommender** (calculates values), **Updater** (evicts Pods that need changes) and an **Admission Controller** (sets requests on new Pods).

updateMode	Behaviour
`"Off"`	Only produce recommendations; change nothing. **Best way to start.**
`"Initial"`	Apply recommendations only when Pods are created
`"Recreate"`	Also evict running Pods so they restart with new values
`"Auto"`	Currently acts like Recreate; may use in-place resizing where supported
`"InPlaceOrRecreate"`	Newer mode: tries to resize without restarting, else falls back to eviction. Check that your VPA and Kubernetes versions support it.

vpa.yaml

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: web-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  updatePolicy:
    updateMode: "Off"             # just recommend
  resourcePolicy:
    containerPolicies:
      - containerName: "*"
        minAllowed: { cpu: 100m, memory: 128Mi }
        maxAllowed: { cpu: "2", memory: 2Gi }
```

`kubectl describe vpa web-vpa` then shows **Lower Bound / Target / Upper Bound** suggestions for each container.

> **Watch out**
> Do **not** let HPA and VPA both act on the **same metric** (CPU or memory) for the same workload: they fight each other. Use VPA for right-sizing (often in `Off` mode) and HPA for scaling out, or let HPA use custom metrics.

13.4 Cluster Autoscaler (CA)

HPA can add Pods, but if no node has room they sit in **Pending**. The **Cluster Autoscaler** watches for exactly that and **adds nodes** (by growing a cloud node group such as an AWS Auto Scaling Group, Azure VMSS or Google managed instance group). It also **removes nodes** that stay under-used, after moving their Pods elsewhere.

- **Scale up trigger:** Pods that are unschedulable because of insufficient CPU/memory.
- **Scale down trigger:** a node whose Pods could all fit on other nodes, below a utilization threshold for a sustained period (defaults are roughly 50% and 10 minutes, configurable).
- **Respects limits:** min/max size per node group; Pods with `PodDisruptionBudgets`, local storage or restrictive affinity can block scale-down.
- **Alternatives: Karpenter** (provisions right-sized nodes directly, popular on AWS) and managed autopilot-style modes from cloud providers.

cluster-autoscaler args (excerpt)

```
# Example Cluster Autoscaler flags (AWS): scale the node group between 1 and 10 nodes
--cloud-provider=aws
--nodes=1:10:my-node-group
--scale-down-utilization-threshold=0.5
--scale-down-unneeded-time=10m
```

> **The scaling chain**
> Load rises -> **HPA** adds Pods -> some Pods are **Pending** -> **Cluster Autoscaler** adds a node -> Pods schedule. Load falls -> HPA removes Pods -> nodes become under-used -> Cluster Autoscaler removes nodes. **VPA** quietly improves the CPU/memory size of each Pod throughout.

14 Putting It All Together

How the 13 topics fit in one real application

A typical production web application uses almost every concept you just learned:

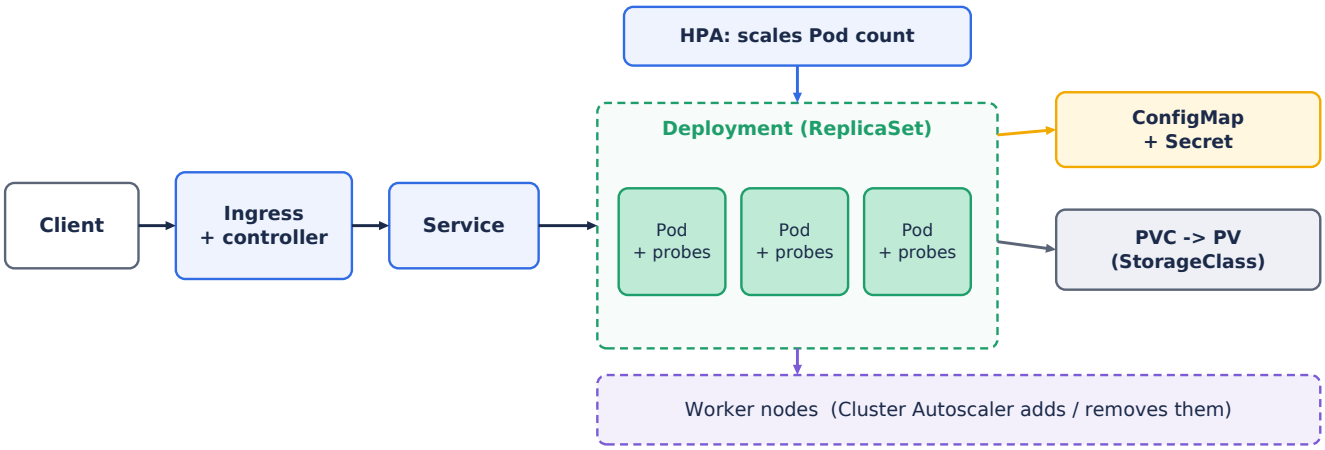


Figure 14.1 - One application, many Kubernetes objects

Need	Use this object	Chapter
Run an app container	Pod (created for you by a controller)	2
Isolate teams / environments	Namespace	3
Group and select objects	Labels + selectors	3
Stable address inside the cluster	Service (ClusterIP)	4
Expose to the internet	LoadBalancer Service, or Ingress / Gateway	4, 12
Stateless app with rolling updates and rollback	Deployment (-> ReplicaSet)	5, 6
Database / clustered app needing stable identity	StatefulSet + headless Service + PVCs	7, 10
Agent on every node (logs, monitoring)	DaemonSet	7
Control where Pods run	Node affinity, taints and tolerations	8
App settings and credentials	ConfigMap and Secret	9
Data that survives restarts	PV, PVC, StorageClass	10
Self-healing and safe rollouts	Startup / Liveness / Readiness probes	11
Handle traffic spikes automatically	HPA (+ VPA, Cluster Autoscaler)	13

Essential kubectl cheat sheet

Task	Command
Apply / delete from a file	`kubectl apply -f x.yaml` / `kubectl delete -f x.yaml`
List objects	`kubectl get pods,svc,deploy -A -o wide`
Inspect and see events	`kubectl describe <kind> <name>`
Logs (follow, previous crash)	`kubectl logs -f <pod>` / `kubectl logs <pod> --previous`
Shell into a container	`kubectl exec -it <pod> -- sh`
Local access to a Pod/Service	`kubectl port-forward svc/<name> 8080:80`
Generate YAML quickly	`kubectl create deployment web --image=nginx --dry-run=client -o yaml`
Explain any field	`kubectl explain deployment.spec.strategy`
Rollouts	`kubectl rollout status|history|undo deployment/<name>`
Resource usage	`kubectl top nodes` / `kubectl top pods`

Quick troubleshooting path

1. `kubectl get pods`: read the STATUS (see the table in Chapter 2).
2. `kubectl describe pod <name>`: read the **Events** at the bottom (scheduling, image pull, probe failures).
3. `kubectl logs <name>` (add `--previous` after a crash).
4. Service not reachable? `kubectl get endpoints <svc>`: an empty list means the **selector does not match** Pod labels or Pods are not Ready.
5. Pod Pending? Check requests vs node capacity, taints/tolerations, affinity and PVC binding.

Suggested practice plan

- **Week 1:** Pods, Deployments, Services, labels. Break things and watch self-healing.
- **Week 2:** ConfigMaps, Secrets, probes, rolling updates and rollbacks. Try a Blue-Green and a Canary by hand.
- **Week 3:** PV/PVC/StorageClass, StatefulSet, DaemonSet, scheduling controls.
- **Week 4:** Ingress or Gateway with TLS, HPA with a load test, then explore Helm and RBAC.

Official references

Always cross-check details against the documentation for the Kubernetes version you run:
Cluster components - https://kubernetes.io/docs/concepts/overview/components/
Pods - https://kubernetes.io/docs/concepts/workloads/pods/
Services - https://kubernetes.io/docs/concepts/services-networking/service/
Deployments - https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
Persistent Volumes - https://kubernetes.io/docs/concepts/storage/persistent-volumes/
Liveness, readiness and startup probes - https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
Ingress - https://kubernetes.io/docs/concepts/services-networking/ingress/
Vertical Pod autoscaling - https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/
Ingress NGINX retirement notice - https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/